

Algoritmi Elementari

Recapitulare Informatică clasa a IX-a (C/C++)

Probleme de Maxim și Minim

Identificarea valorilor extreme într-un set de date.

Principiul de Maxim

Pentru a găsi maximul dintre mai multe numere (ex: **a**, **b**, **c**), nu trebuie să comparăm fiecare număr cu toate celelalte simultan.

Logica algoritmică (Metoda presupunerii):

1. Presupunem că **primul număr** este maximul.
2. Comparăm al doilea număr cu maximul. Dacă este mai mare, devine noul maxim.
3. Repetăm procesul pentru toate numerele rămase.

```
// Exemplu: Maximul dintre a, b și c
int a = 15, b = 42, c = 8;

// Pas 1: Presupunem că 'a' este cel mai mare
int maxim = a;

// Pas 2: Actualizăm dacă găsim ceva mai mare
if (b > maxim) {
    maxim = b;
}
if (c > maxim) {
    maxim = c;
}

cout << "Maximul este: " << maxim;
```

Maximul unui Șir

Când procesăm un șir de **N** numere, procedăm similar, folosind o structură repetitivă (**for** sau **while**).

- Citim numărul **N** (câte numere procesăm).
- Citim **primul număr** separat și îl setăm ca maxim.
- Citim restul de **N-1** numere într-o buclă, actualizând maxim la fiecare pas dacă este necesar.

```
int n, x, maxim;
cin >> n;

// Citim primul element
cin >> x;
maxim = x;

// Procesăm restul de n-1 elemente
for (int i = 1; i < n; i++) {
    cin >> x;
    if (x > maxim) {
        maxim = x;
    }
}
cout << "Valoarea maximă citită: " << maxim;
```

Cifrele unui Număr

Extragerea și prelucrarea structurii interne a numerelor întregi.

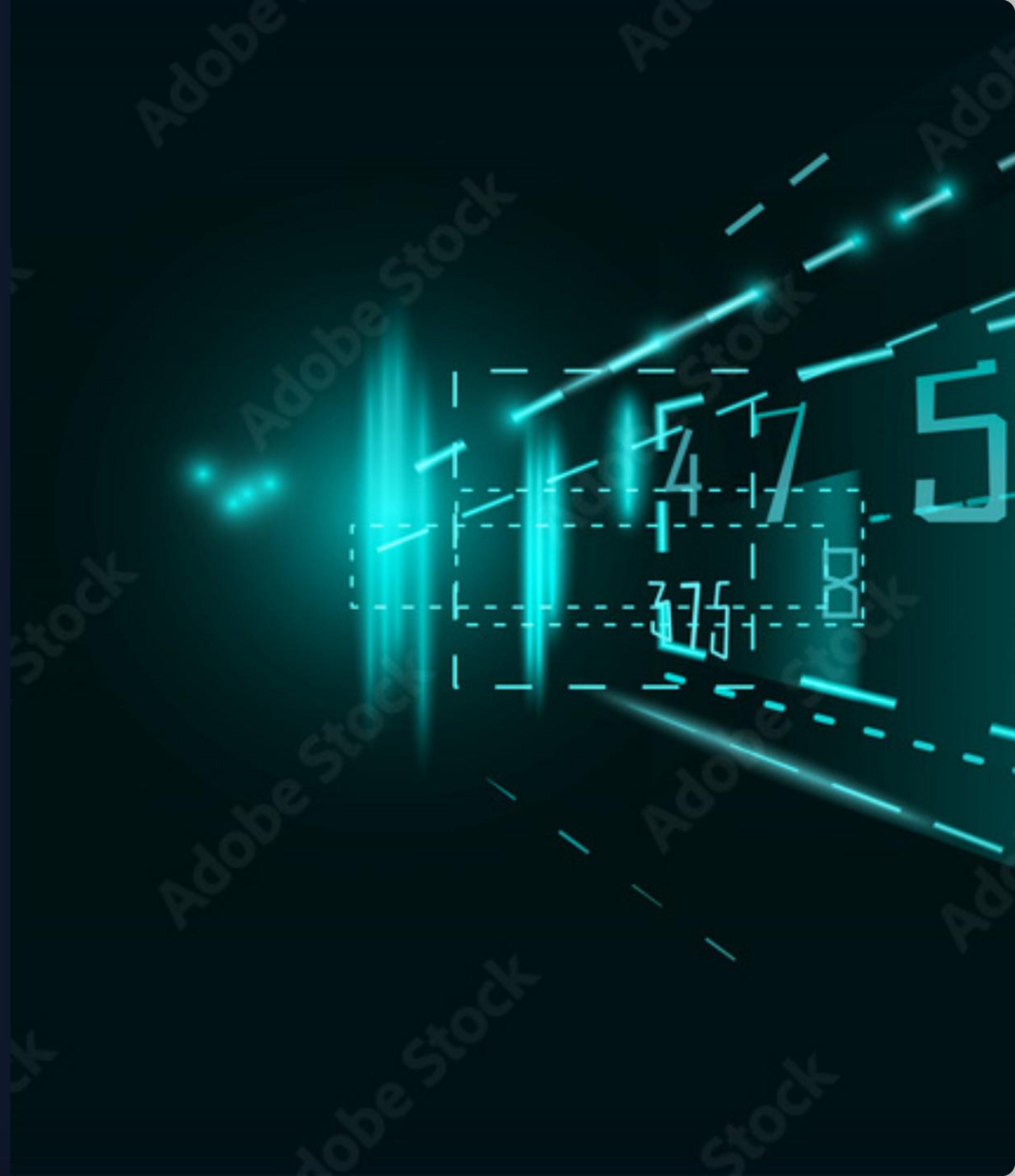
Logica Extragerii

Totul se bazează pe Baza 10.

Pentru a dezasambla un număr cifră cu cifră (de la dreapta la stânga), folosim două operații matematice vitale în C++:

Modulo (% 10): Ne oferă întotdeauna ultima cifră a numărului.

Împărțirea (/ 10): Elimină ultima cifră. În C++, împărțirea întreagă trunchiază automat partea zecimală.



Probleme Clasice



Suma Cifrelor

La fiecare pas extragem ultima cifră cu $\% 10$, o adunăm la o variabilă inițializată cu 0, apoi eliminăm cifra folosind $/ 10$.



Numărul de Cifre

În loc să adunăm valoarea cifrei, pur și simplu incrementăm un contor (ex: $k++$) de fiecare dată când tăiem o cifră, până când numărul devine 0.



Oglinditul unui Număr

Inversul se construiește iterativ:
 $ogl = ogl * 10 + cifra$.
Aceasta mută cifrele vechi la stânga și o adaugă pe cea nou extrasă.

Algoritmul în C++

Orice prelucrare a cifrelor unui număr n folosește un șablon standard cu o buclă **while**.

Condiția de oprire este $n > 0$ (sau $n \neq 0$).

Atenție: Dacă ai nevoie de valoarea inițială a lui n mai târziu în program, creează o copie (ex: `int copie_n = n;`) înainte de a intra în buclă, deoarece n va deveni 0.

```
int n, suma = 0, ogl = 0;
cin >> n;

while (n > 0) {
    int cifra = n % 10; // Extragem ultima cifră

    suma += cifra; // Adunăm la sumă
    ogl = ogl * 10 + cifra; // Construim oglinditul

    n /= 10; // Tăiem cifra procesată
}

cout << "Suma: " << suma << endl;
cout << "Oglinditul: " << ogl << endl;
```

Divizorii unui Număr

Găsirea divizorilor, numere prime și optimizări matematice.

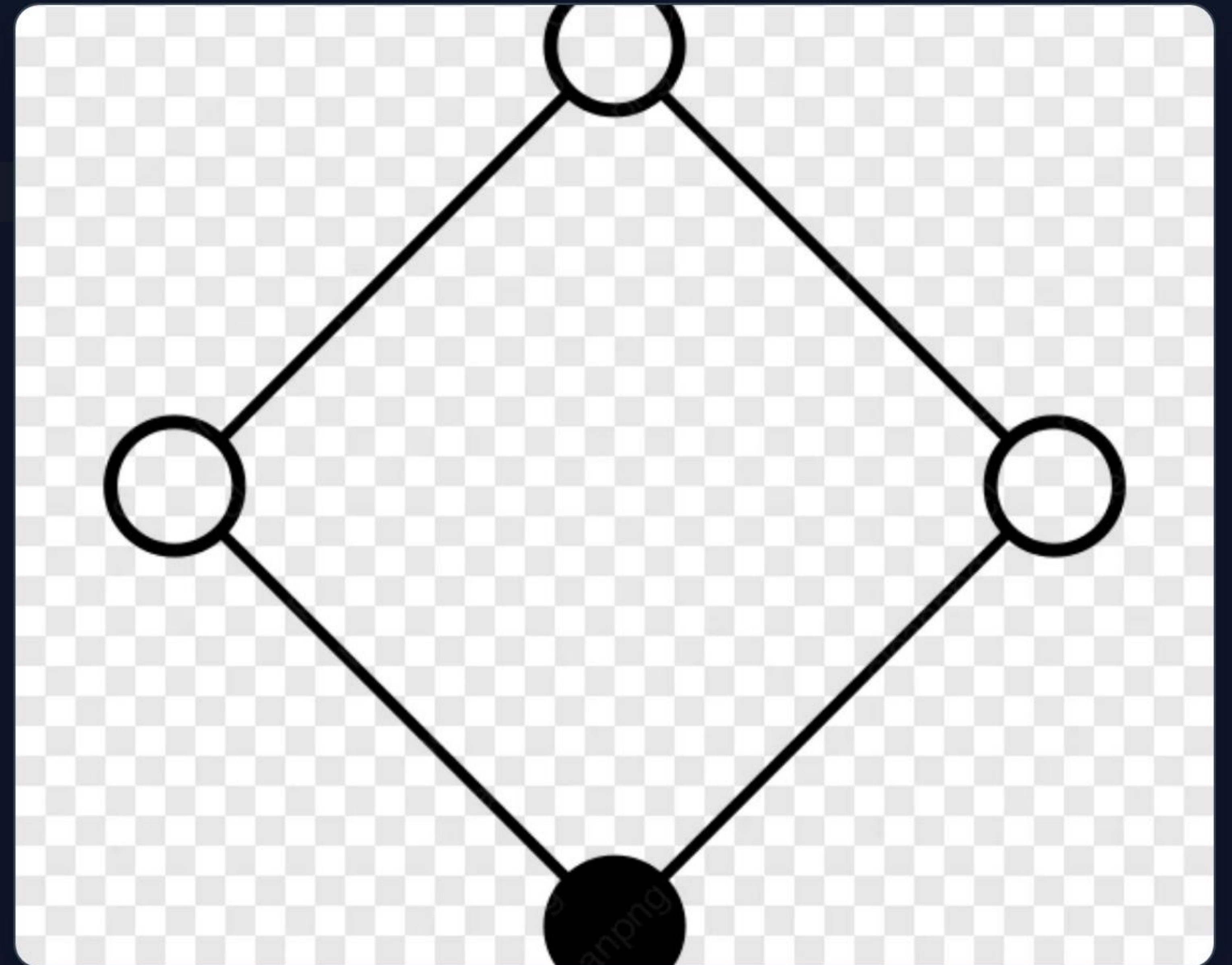
Optimizarea Divizorilor

Algoritmul naiv verifică toate numerele de la 1 la n . Acest lucru este extrem de lent pentru numere foarte mari (ex: 1 miliard).

Observație cheie: Divizorii vin mereu în perechi (ex: pentru 36, avem perechea 4 și 9). Cel mai mic număr dintr-o pereche nu poate depăși radicalul lui n .

Prin urmare, este suficient să căutăm divizori până la:

$$d \leq \sqrt{n}$$



Algoritmul Optimizat

Acest algoritm parcurge numerele d de la 1 atâta timp cât $d * d \leq n$ (echivalent cu $d \leq \sqrt{n}$).

Când găsim un divizor d , afișăm automat și **perechea sa**, care este n / d .

Condiția `if (d != n / d)` ne protejează de afișarea aceluiasi divizor de două ori în cazul pătratelor perfecte (cum ar fi 6 și 6 pentru numărul 36).

```
int n;
cin >> n;

cout << "Divizorii sunt: " << endl;
for (int d = 1; d * d <= n; d++) {
    if (n % d == 0) {
        cout << d << " "; // Primul divizor
        if (d != n / d) {
            cout << n / d << " "; // Perechea sa
        }
    }
}
```

Testul de Primalitate

Un număr este **prim** dacă se împarte exact doar la 1 și la el însuși. (0 și 1 NU sunt prime).

Pentru verificare, presupunem că numărul este prim (`bool estePrim = true;`). Apoi căutăm orice divizor între 2 și radicalul lui n .

Dacă găsim cel puțin un divizor, numărul NU este prim, așa că setăm `estePrim = false` și oprim căutarea prematur folosind **break**.

```
int n; cin >> n;
bool estePrim = true;

if (n < 2) estePrim = false;

for (int d = 2; d * d ≤ n; d++) {
    if (n % d == 0) {
        estePrim = false; // Am găsit un divizor
        break; // Oprim căutarea
    }
}

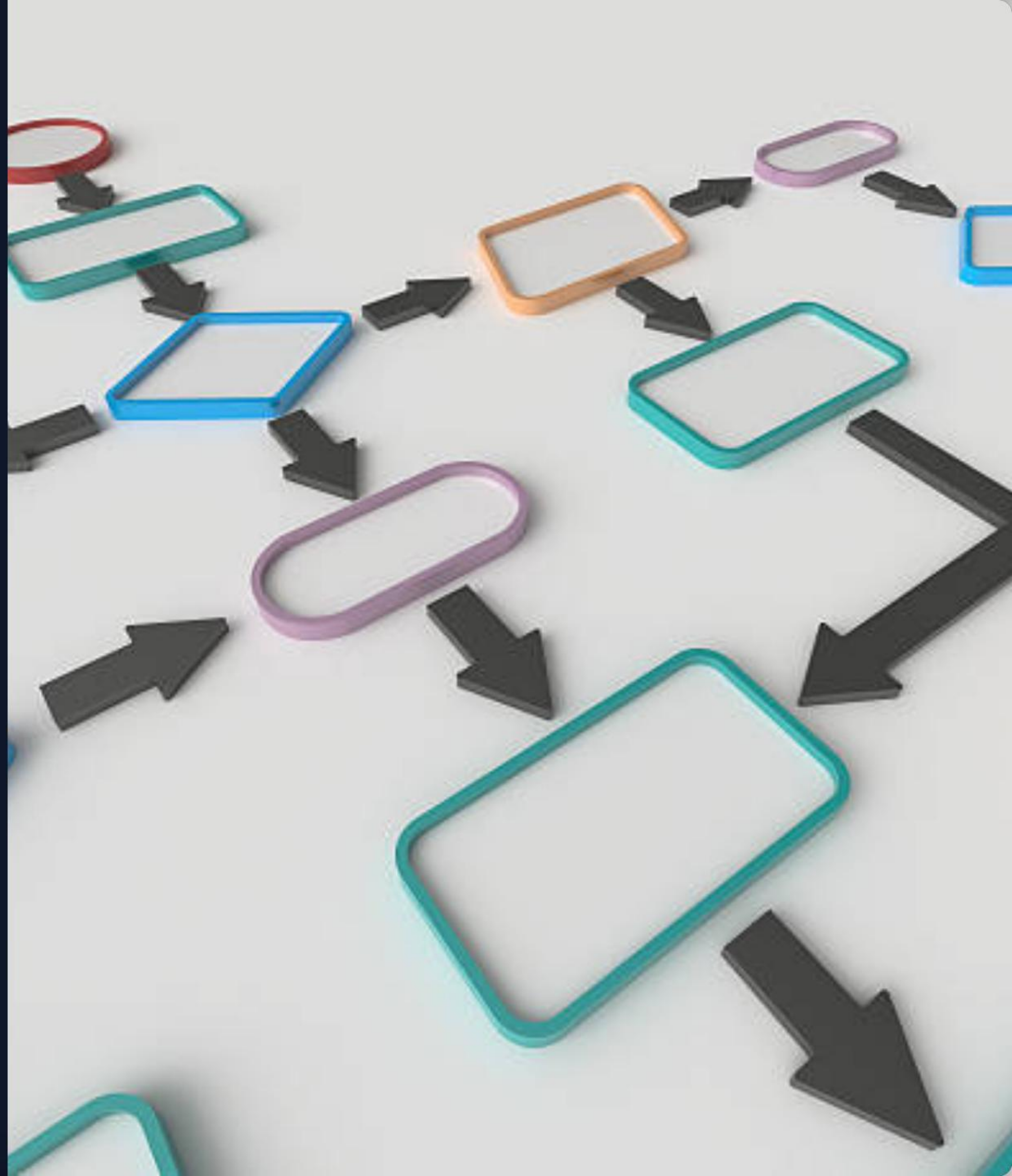
if (estePrim) cout << "Prim";
else cout << "Nu este prim";
```

Descompunerea în Factori Primi

Teorema Fundamentală a Aritmeticii.

Orice număr compus poate fi scris ca un produs unic de numere prime ridicate la anumite puteri.

Algoritmul împarte repetat numărul la cel mai mic divizor posibil (începând cu 2) până când nu se mai poate împărți. Apoi trecem la următorul divizor, repetând procesul până când n devine 1.



Cod: Descompunere

Nu este nevoie să verificăm dacă d este prim! Buclo interioară (`while (n % d == 0)`) consumă toți factorii lui d . Când ajungem la un multiplu al lui d (cum ar fi 4 sau 6), numărul n a fost deja împărțit la factorii primi (2, 3) și nu se va mai divide exact.

Linia `if (d * d > n)` este o **optimizare crucială** pentru a evita parcurgerea numerelor inutile până la un ultim factor prim foarte mare.

```
int n; cin >> n;
int d = 2;
while (n > 1) {
    int p = 0; // Puterea factorului d
    while (n % d == 0) {
        p++;
        n /= d;
    }

    if (p > 0) {
        cout << d << "^" << p << endl;
    }

    d++;
    if (d * d > n && n > 1) {
        d = n; // Optimizare pt. ultimul factor mare
    }
}
```

Întrebări?

Sunteți gata să rezolvați probleme în C++!

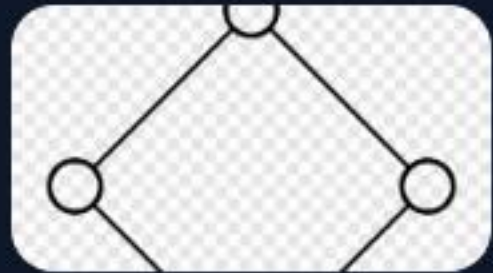
Succes la implementare!

Image Sources



https://as1.ftcdn.net/v2/jpg/04/09/90/08/1000_F_409900893_LN2AZzKtCMUILXoWcivrsv79iHROyH4A.jpg

Source: www.europosters.pt



<https://banner2.cleanpng.com/20180625/iaa/kisspng-mathematical-problem-node-mathematics-tree-edge-co-30-minutes-5b31494425c1a8.3512494115299566761547.jpg>

Source: www.cleanpng.com



<https://media.istockphoto.com/id/490245946/photo/flow-chart-diagram-concept.jpg?s=612x612&w=0&k=20&c=SWE8UA7W8oBrMYO2VbXcxJjSdew-Gjl9v5zKknJtl4U=>

Source: www.istockphoto.com